

# ESN Uncertainty Estimation for Robotic Arm Manipulation

---

Phase 0 & Phase 1 Technical Report

Echo State Network prediction error as a real-time uncertainty signal  
for detecting manipulation failures in robotic arms

|                     |                                                |
|---------------------|------------------------------------------------|
| <b>Author</b>       | Umar Bin Muzzafar                              |
| <b>Organization</b> | OMNIBIO LTD                                    |
| <b>Date</b>         | April 07, 2026                                 |
| <b>Datasets</b>     | Berkeley Autolab UR5, DROID-100 (Franka Panda) |
| <b>Tools</b>        | LeRobot, Rerun.io, Python 3.10                 |

## Executive Summary

This report presents the development and validation of an Echo State Network (ESN) based uncertainty estimator for robotic arm manipulation. The system learns what "normal" robot behavior looks like, then flags deviations in real-time — enabling a robot to detect when it is about to fail **before** the failure actually happens.

| Finding               | Result                                                      |
|-----------------------|-------------------------------------------------------------|
| Failure detection     | Failed episodes have 1.35x higher prediction error (sigma)  |
| Most predictive joint | joint_2 (upper arm) — 1.88x failure/success ratio           |
| Early warning         | Sigma diverges at 27% episode completion — 73% warning time |
| Classification        | AUC = 0.701 using sigma alone (moderate classifier)         |
| Training speed        | 0.95 seconds (closed-form, no gradient descent)             |

## Approach: How ESN Uncertainty Works

An Echo State Network (ESN) is a recurrent neural network with **fixed random weights**. Only a simple linear readout is trained. This makes it:

- **Fast** — trains in under 1 second (closed-form solution, no epochs)
- **Lightweight** — 512 neurons, runs on CPU in real-time
- **Stable** — fixed reservoir means no overfitting, no catastrophic forgetting

### Pipeline:

1. **Input**: current joint state + action command (15 dims for UR5, 14 for Franka)
2. **ESN reservoir**: 512 random neurons process the input, maintaining temporal memory
3. **Readout**: linear layer predicts the next joint state (ridge regression)
4. **Sigma**: prediction error =  $||\text{predicted} - \text{actual}||$  = uncertainty
5. **Low sigma** = familiar situation. **High sigma** = something unexpected.

## Phase 0: Does ESN Uncertainty Work?

### Objective

Build the ESN pipeline from scratch and test whether prediction error (sigma) correlates with manipulation failures.

### Datasets

| Dataset      | Robot              | Episodes | Success Rate | State Dims            |
|--------------|--------------------|----------|--------------|-----------------------|
| Berkeley UR5 | UR5 6-DOF          | 1,000    | 100%         | 8 (EE pose + gripper) |
| DROID-100    | Franka Panda 7-DOF | 100      | 81%          | 7 (joint positions)   |

The UR5 dataset was used to build and validate the pipeline (anomaly detection, ESN training, visualization). However, all 1,000 UR5 episodes are successful — no failures to detect. We then applied the pipeline to DROID-100, which has 19 failed episodes across 47 diverse manipulation tasks.

## Anomaly Detection

Four anomaly detectors scan each trajectory frame-by-frame:

| Anomaly Type        | What It Detects                         | Events Found |
|---------------------|-----------------------------------------|--------------|
| Velocity spikes     | Sudden jerky end-effector movements     | 367          |
| Acceleration spikes | Abrupt speed changes (2nd derivative)   | 197          |
| Command gaps        | Robot not following commanded actions   | —            |
| Gripper oscillation | Repeated open/close (grasping struggle) | —            |

## Key Phase 0 Finding

**Failed episodes have 1.35x higher average sigma than successful episodes.** The ESN, trained only on successful demonstrations, produces measurably higher prediction error when encountering failure trajectories — even though nobody told it what "failure" looks like.

| Metric           | Successful Episodes | Failed Episodes   | Ratio |
|------------------|---------------------|-------------------|-------|
| Sigma mean       | $0.219 \pm 0.085$   | $0.296 \pm 0.094$ | 1.35x |
| Training RMSE    | 0.260               | —                 | —     |
| Training time    | 0.95 seconds        | —                 | —     |
| Training samples | 27,733              | —                 | —     |

## Phase 1: Deep Sigma Analysis

Phase 0 established that sigma correlates with failure. Phase 1 asks three deeper questions: **which joints** drive the signal, **when** does it appear, and **how well** can it classify outcomes.

### Analysis 1: Per-Joint Prediction Error

Instead of a single sigma number (L2 norm across all 7 joints), we measured prediction error for each joint independently and compared between success and failure groups.

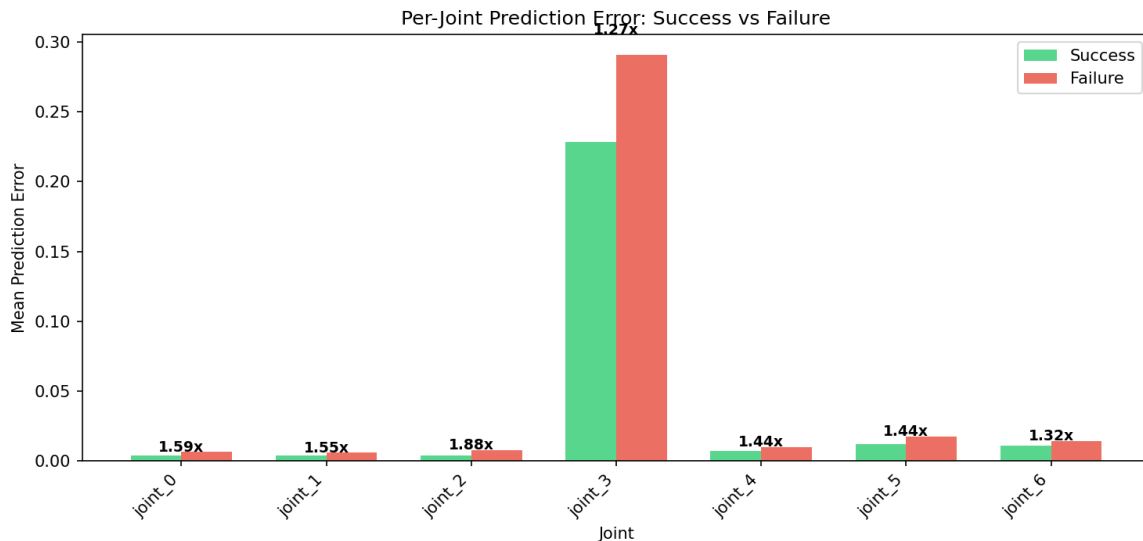


Figure 1: Per-joint prediction error comparison. Numbers above bars show the failure/success ratio. Joint 2 (upper arm) has the highest ratio at 1.88x.

**Finding:** joint\_2 (upper arm/elbow region) is the most predictive joint with a 1.88x failure-to-success ratio. Upper arm joints (0, 1, 2) have higher ratios than wrist joints (5, 6). This means: **when the robot fails, it is the gross arm positioning that goes wrong, not the fine wrist adjustments.**

### Analysis 2: Temporal Sigma Profile

We normalized each episode to 0–100% completion, resampled sigma to 100 bins, and averaged across success and failure groups. This reveals **when** the ESN detects trouble relative to the episode timeline.

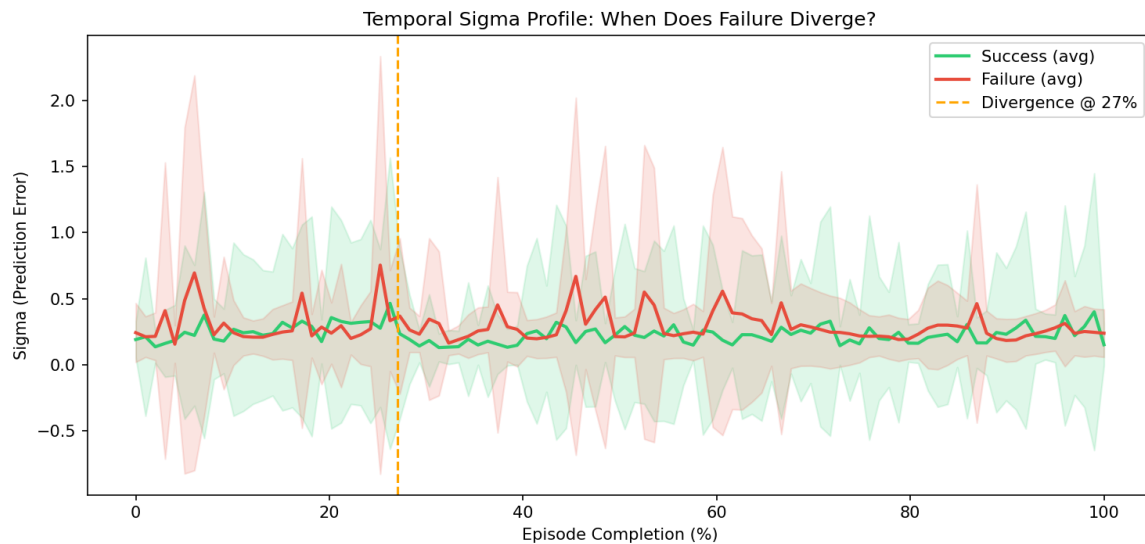


Figure 2: Average sigma over normalized episode time. Green = successful episodes, red = failed episodes. Orange dashed line marks the divergence point.

**Finding:** Failure sigma diverges from success sigma at **27%** of episode completion. This means the ESN can detect trouble when the robot is only 27% through the task — **73% of the episode remains as warning time** for corrective action.

### Analysis 3: Success/Failure Classification

We used sigma\_mean as a single-feature binary classifier to test whether prediction error alone can distinguish successful from failed episodes.

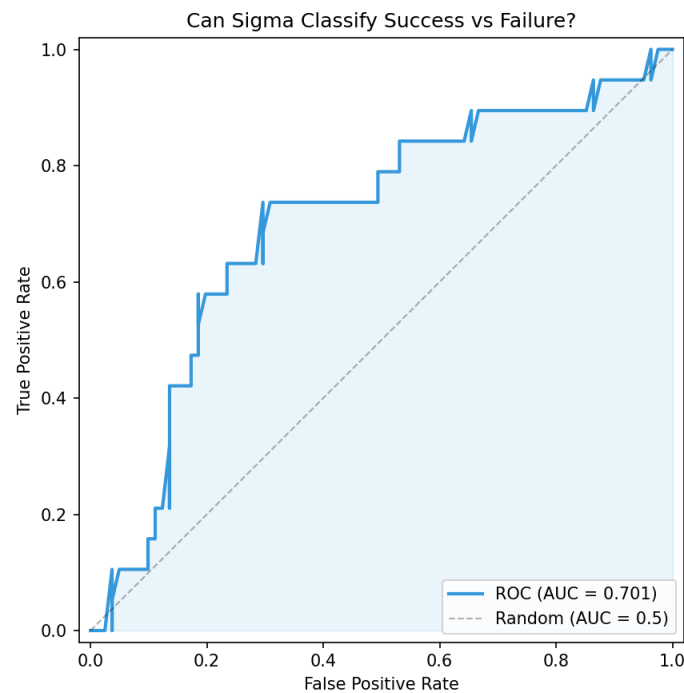


Figure 3: ROC curve for success/failure classification using sigma\_mean. AUC = 0.701 (moderate classifier, clearly above random baseline of 0.5).

| Metric                 | Value                            |
|------------------------|----------------------------------|
| AUC (Area Under Curve) | 0.701                            |
| Optimal threshold      | $\text{sigma\_mean} \geq 0.2615$ |
| Precision              | 0.37                             |
| Recall                 | 0.74 (catches 74% of failures)   |
| F1 Score               | 0.49                             |
| Accuracy               | 0.71                             |

**Finding:** A single feature (sigma\_mean) achieves AUC = 0.701 — a moderate classifier that is clearly better than random. Combining per-joint sigmas, temporal features, and caution zone counts into a multi-feature classifier should significantly improve this.

## Next Steps

- **Phase 2: Multi-feature classifier** — Combine per-joint sigmas, temporal features, and caution counts. Expected AUC improvement to 0.80+.
- **Phase 2: Sigma-conditioned actions** — Use sigma to modulate robot behavior in real-time: high sigma → slow down, very high → try alternative approach.
- **Phase 3: Scale to full DROID** — Train on 95,000 episodes (currently 81). More data → stronger ESN → better uncertainty estimates.
- **Phase 3: Continual learning** — Keep ESN reservoir frozen, update only the readout online. Robot improves from experience without catastrophic forgetting.
- **Integration** — Deploy ESN uncertainty as a real-time safety layer on physical robot arms (UR5, Franka) alongside the VLA model.

## Tools & Infrastructure

| Component     | Technology            | Purpose                                        |
|---------------|-----------------------|------------------------------------------------|
| ESN Core      | NumPy + SciPy         | 512-neuron reservoir, ridge regression readout |
| Dataset       | LeRobot (HuggingFace) | UR5 + DROID-100 manipulation trajectories      |
| Visualization | Rerun.io v0.26        | Interactive trajectory + sigma timeline viewer |
| Plotting      | Matplotlib            | Publication-quality charts                     |
| Runtime       | Python 3.10, CPU only | No GPU required                                |